

---

# CS 61B

## Spring 2025

Midterm 1

Thursday, February 20, 2025

---

Your Name: \_\_\_\_\_

Your SID: \_\_\_\_\_ Location: \_\_\_\_\_

SID of person to your left: \_\_\_\_\_ Right: \_\_\_\_\_

This exam is worth **100 points**, and consists of 8 questions. Questions are ordered by topic, not by difficulty.

|           |    |    |    |    |    |       |
|-----------|----|----|----|----|----|-------|
| Question: | 1  | 2  | 3  | 4  | 5  | Total |
| Points:   | 12 | 24 | 29 | 20 | 15 | 100   |

- ☐ indicates only one circle should be filled in. ☐ indicates more than one box may be filled in. **Please fill in the shape completely.** If you change your response, **erase as completely as possible.**
- Anything you write that you ~~cross out~~ will not be graded.
- If you write multiple answers, your answer is ambiguous, or the bubble/checkbox is not entirely filled in, we will grade according to the worst interpretation.
- Unless otherwise specified, all data structures and algorithms behave according to their implementation in lecture, with no additional optimizations.
- If an implementation detail (e.g. tiebreaking scheme, linked list topology) is relevant, it will be explicitly noted in the question.

Coding questions:

- You may write at most one statement per blank and you may not use more blanks than provided.
- Your answer will be reformatted according to the 61B style guidelines. For example, any if-statement requires at least three lines for the purposes of determining line count.
- You may not use ternary operators, lambdas, streams, or multiple assignment.
- Unless otherwise specified, you may assume that everything on the reference card has been imported.

Write the statement below in the same handwriting you will use on the rest of the exam. Failure to do so may result in a voided exam.

|                                                                                                                                                                                               |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| I have neither given nor received help on this exam (or quiz), and have rejected any attempt to cheat.<br>If these answers are not my own work, I may be deducted up to 9,876,543,210 points. |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

---

---

---

---

---

Signature: \_\_\_\_\_

## 1 CS61Bot

(12 points)

The ChatBot interface is defined below:

```
public interface ChatBot {
    public void answerQuery(String q);
    default public String myInfo() {
        return "I am a ChatBot!";
    }
}
```

(a) (7.5 points) Suppose we create a class CS61Bot that implements ChatBot. Which of the following are true? Select all that apply.

- ☐ The CS61Bot class can compile even if it does not define an answerQuery method.
- ☐ The CS61Bot class can compile even if it does not define a myInfo method.
- ☐ If we override myInfo, we must add an @Override tag in order for the CS61Bot class to compile.
- ☐ If we do not override answerQuery, our program may crash at runtime.
- ☐ We can define a constructor for CS61Bot even though the interface doesn't list any constructors.

(b) (1.5 points) The CS61Bot class is defined below:

```
public class CS61Bot implements ChatBot {
    public static String versionInfo = "CS61Bot version 5";
    @Override
    public void answerQuery(String q) {
        // implementation not shown
    }
    @Override
    public String myInfo() {
        return versionInfo;
    }
    public static void main(String[] args) {
        CS61Bot bot = new CS61Bot();
        System.out.println(bot.myInfo());
    }
}
```

What happens when the main method is executed?

- ☐ The program prints "I am a ChatBot!".
- ☐ The program prints "CS61Bot version 5".
- ☐ The program errors at compile-time.
- ☐ The program errors at run-time.
- ☐ None of the above.

Aditya wants to add a new method `public void answerQuery(String q, String student)` to the `CS61Bot` class.

- (c) (1.5 points) What happens if Aditya **adds** the method to the class (without removing the existing `answerQuery` method)?
- ☐ The `CS61Bot` class will not compile because it defines two methods with the same name.
  - ☐ The `CS61Bot` class will not compile for some other reason.
  - ☐ The `CS61Bot` class will compile.
- (d) (1.5 points) What happens if Aditya **replaces** the method in the class (removing the existing `answerQuery` method)?
- ☐ The `CS61Bot` class will not compile as it does not implement all methods of the `ChatBot` interface.
  - ☐ The `CS61Bot` class will compile, and attempting to call `answerQuery(q)` (where `q` is a `String`) will call `answerQuery(q, null)` instead.
  - ☐ The `CS61Bot` class will compile, but attempting to call `answerQuery(q)` (where `q` is a `String`) will cause a runtime error.
  - ☐ None of the above.

## 2 2048 Comparator

(24 points)

Given a row of a 2048 board, define the `rowwiseDropCount` of a row as the number of elements in the row which are strictly less than the value immediately before them (going within the row from left to right). For example:

- [128, 64, 32, 16] has a **rowwiseDropCount of 3**:  $128 > 64$ ,  $64 > 32$ ,  $32 > 16$ .
- [128, 4, 32, 0] has a **rowwiseDropCount of 2**:  $128 > 4$ ,  $32 > 0$ .
- [8, 0, 0, 2] has a **rowwiseDropCount of 1**:  $8 > 0$ .

The `columnwiseDropCount` is defined similarly, going within a column from top to bottom.

Define the `totalDropCount` as the sum of the `rowwiseDropCounts` and `columnwiseDropCounts` of all rows and columns. For example:

```
// totalDropCount(board1) = (3 + 3 + 3 + 3) + (3 + 3 + 3 + 3) = 24
//                      ^ rows ^           ^ columns ^
```

```
int[][] board1 = {
    {128, 64, 32, 16},
    { 64, 32, 16, 8},
    { 32, 16, 8, 4},
    { 16, 8, 4, 2}
};
```

```
// totalDropCount(board2) = (3 + 1 + 1 + 3 + 2) + (3 + 1 + 2 + 1 + 3) = 20
//                      ^ rows ^           ^ columns ^
```

```
int[][] board2 = {
    {128, 4, 32, 2, 0}, // 3 drops in this row: 128 to 4, 32 to 2, and 2 to 0
    { 4, 32, 64, 8, 64}, // 1 drop in this row: 64 to 8
    { 16, 64, 128, 16, 32}, // 1 drop in this row: 128 to 16
    { 8, 4, 8, 4, 2} // 3 drops in this row: 8 to 4, 8 to 4, and 4 to 2
    { 2, 4, 0, 8, 0} // 2 drops in this row: 4 to 0 and 8 to 0
};
```

// In board2, we have 3 drops in the first column, 1 drop in the second column, 2 drops in the third column, 1 drop in the fourth column, and 3 drops in the fifth column

For this question, you may assume that all boards are square.

- (a) (8 points) Implement the `totalDropCount` method. For your convenience, the examples from the previous page are shown below.

```
// totalDropCount(board1) = (3 + 3 + 3 + 3) + (3 + 3 + 3 + 3) = 24
//           ^ rows ^           ^ columns ^
int[][] board1 = {
    {128, 64, 32, 16},
    { 64, 32, 16,  8},
    { 32, 16,  8,  4},
    { 16,  8,  4,  2}
};

// totalDropCount(board2) = (3 + 1 + 1 + 3 + 2) + (3 + 1 + 2 + 1 + 3) = 20
//           ^ rows ^           ^ columns ^
int[][] board2 = {
    {128,  4, 32,  2,  0}, // 3 drops in this row: 128 to 4, 32 to 2, and 2 to 0
    {  4, 32, 64,  8, 64}, // 1 drop in this row: 64 to 8
    { 16, 64, 128, 16, 32}, // 1 drop in this row: 128 to 16
    {  8,  4,  8,  4,  2} // 3 drops in this row: 8 to 4, 8 to 4, and 4 to 2
    {  2,  4,  0,  8,  0} // 2 drops in this row: 4 to 0 and 8 to 0
}; // Columns have 3, 1, 2, 1, and 3 drop(s) respectively

private static int totalDropCount(int[][] board) {
    int count = 0;
    for (int i = 0; _____; i += 1) {
        1

        for (int j = _____; _____; j += 1) {
            2                3

            if (_____ ) {
                4
                count += 1;
            }

            if (_____ ) {
                5
                count += 1;
            }
        }
    }
    return count;
}
```

- (b) (5 points) Implement the findMax method, which finds the largest tile value in board.

```
private static int findMax(int[][] board) {
    int maxTile = 0;

    for (_____ 1 _____ row : board) {

        for (_____ 2 _____ tile : row) {

            if (_____ 3 _____) {
                maxTile = tile;
            }
        }
    }
    return maxTile;
}
```

- (c) (11 points) Implement BoardComparator, which compares boards by their largest tile value; if equal, the boards' totalDropCounts are used as a tiebreaker. You may assume that parts a and b have been implemented correctly.

```
public static class BoardComparator implements _____ 1 _____ {

    private static int totalDropCount(int[][] board) {
        // implemented in part a, you may assume this works correctly
    }

    private static int findMax(int[][] board) {
        // implemented in part b, you may assume this works correctly
    }

    @Override
    public int compare(int[][] boardA, int[][] boardB) {

        int tileA = _____ 2 _____;

        int tileB = _____ 3 _____;

        if (_____ 4 _____) {

            int scoreA = _____ 5 _____;

            int scoreB = _____ 6 _____;

            return _____ 7 _____;
        }
        return _____ 8 _____;
    }
}
```

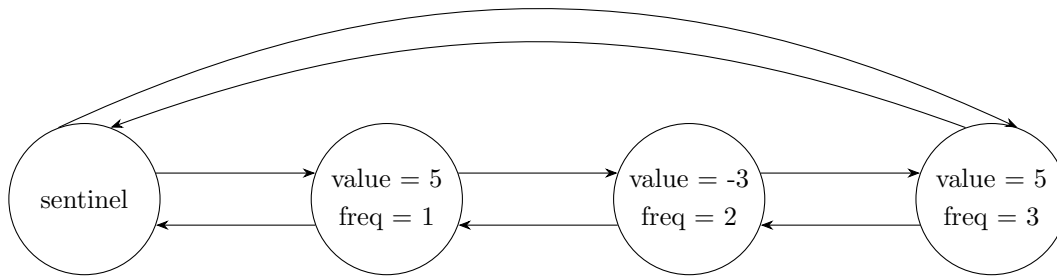
### 3 Compressed Doubly-Linked List

(29 points)

Creating new nodes for consecutive identical elements in a doubly-linked list is inefficient! To combat this, 61B Inc. has created a new data structure called the Compressed Doubly-Linked List (CDLL).

Each node in a CDLL has a `next` and `prev` pointer, along with parameters `value` and `freq`, and represents a sequence of `freq` consecutive elements equal to `value`.

For example, a CDLL representing the list `[5, -3, -3, 5, 5, 5]` would look like:



Except for the sentinel, a CDLL may not have two consecutive nodes with the same `value`, and may not have nodes with `freq` less than 1.

The class is defined as follows:

```

public class CDLL implements Iterable<Integer> {
    private Node sentinel;
    private class Node {
        Node prev;
        Node next;
        int value;
        int freq;
        public Node (Node prev, Node next, int value, int freq) {
            this.prev = prev;
            this.next = next;
            this.value = value;
            this.freq = freq;
        }
    }
    public CDLL(List<Integer> original) {
        // part a
    }
    @Override
    public Iterator<Integer> iterator() {
        return new CDLLIterator();
    }
    private class CDLLIterator implements Iterator<Integer> {
        // part b
    }
}

```

- (a) (17 points) Fill out the constructor of the CDLL class, which takes in a `List` as an argument and converts it into its corresponding CDLL representation. You may not need all lines.

```

public CDLL(List<Integer> original) {

    sentinel = new Node(null, null, -1, -1);
    Node curr = sentinel;

    for (_____ : _____) {

        if (_____ && _____) {

            _____;

        } else {

            _____;

            _____;

            _____;

        }
    }

    _____;

    _____;

}

```



- (b) (12 points) Complete the `CDLLIterator` class, whose iteration returns the values of the CDLL in the order they would appear in a regular `List`. Your iterator **may not** modify the CDLL. You may not need all lines.

```

public class CDLLIterator implements Iterator<Integer> {

    Node curr = _____;
                                     1
    int count = 0;

    @Override
    public boolean hasNext() {

        return _____;
                                     2
    }

    @Override
    public Integer next() {

        _____;
                                     3
        _____;
                                     4

        if ( _____ ) {
                                     5
            _____;
                                     6
            _____;
                                     7
        }

        return _____;
                                     8
    }
}

```

## 4 LargeFriend

(20 points)

Fill in the code for the Comparator `LargeFriend`. Its constructor should take in as input a `Comparator<T>` `wrappedComparator`, which tells the `LargeFriend` how to compare two items of type `T`. It should also be possible to iterate over the `LargeFriend`, which will yield all items that have ever been judged as “larger” by the `LargeFriend` object. Consider an example:

```
Comparator<String> c = new AlphabeticalOrderComparator();
LargeFriend<String> largeFriend = new LargeFriend<>(c);

largeFriend.compare("apple", "zebra");    // "zebra" is larger
largeFriend.compare("apple", "apple");    // neither is larger
largeFriend.compare("apple", "banana");   // "banana" is larger
largeFriend.compare("banana", "zebra");   // "zebra" is larger

// Will print: zebra banana zebra
for (String word : largeFriend) {
    System.out.print(word + " ");
}

largeFriend.compare("warthog", "vanessa"); // "warthog" is larger
largeFriend.compare("fish", "chips");      // "fish" is larger

// Will print: zebra banana zebra warthog fish
for (String word : largeFriend) {
    System.out.print(word + " ");
}
```

Implement the `LargeFriend` class. You may not need all lines. **You may only call `wrappedComparator`'s `compare` method once.**

[illegible]

## 5 Consume and Interleave

(15 points)

Consider the ExamDeque interface and ExamArrayDeque class below, which behaves like a Deque with an additional method consumeAndInterleave.

```
public interface ExamDeque<T> {
    void addFirst(T x);
    void addLast(T x);
    List<T> toList();
    boolean isEmpty();
    int size();
    T removeFirst();
    T removeLast();
    default void consumeAndInterleave(ExamDeque<T> other) {
        // Implemented in part b
    }
}

public class ExamArrayDeque<T> implements ExamDeque<T> {
    ...
    public ExamArrayDeque<T>(List<T> elements) {
        // creates an ExamArrayDeque of elements
        ...
    }

    @Override
    public boolean equals(Object o) {
        // Returns true if o is an ExamDeque and contains the same items (in the same order) as this.
        ...
    }
}
```

The consumeAndInterleave method within the ExamDeque interface receives as input an ExamDeque other, which is the same length as, but not == to, the current ExamDeque. It interleaves other with the current ExamDeque **without creating a new ExamDeque**. other should be empty after the call completes.

For example, if we have ExamDeque x = [2, 3, 4, 5, 6] and ExamDeque y = [20, 30, 40, 50, 60], after we call x.consumeAndInterleave(y), x should be [2, 20, 3, 30, 4, 40, 5, 50, 6, 60], and y should be empty.

For both parts below, **you may not need all lines.**

- (a) (5 points) Write a unit test that checks the correctness of `consumeAndInterleave`. An example of how to use the `containsExactly` and `isEqualTo` methods of the Truth library are provided below.

`@Test`

`public boolean consumeAndInterleaveTest() {`

```

_____ 1 _____ x = new ExamArrayDeque(List.of(2, 3, 4));
assertThat(x.toList()).containsExactly(2, 3, 4).inOrder(); // Returns true
assertThat(x).isEqualTo(new ExamArrayDeque(List.of(2, 3, 4))); // Returns true

_____ 2 _____;

_____ 3 _____;

_____ 4 _____;

_____ 5 _____;

_____ 6 _____;

_____ 7 _____;
}

```

- (b) (10 points) Fill in the `consumeAndInterleave` method below in the `ExamDeque` interface.

```

default void consumeAndInterleave(ExamDeque<T> other) {
    if (other.size() != this.size() || this == other) {
        throw new IllegalArgumentException();
    }
}

```

```

_____ 1 _____

_____ 2 _____

_____ 3 _____

_____ 4 _____

_____ 5 _____

_____ 6 _____

_____ 7 _____

}

```

Nothing on this page is worth any points.

---

## BigTest

**(0 Points)**

Write a coverage test for `consumeAndInterleave` that confirms the method works on very long inputs. Your test should `consumeAndInterleave` `ExamDeque`s containing at least 1000 distinct elements.

## Feedback

**(0 Points)**

Leave any feedback, comments, concerns, or more drawings below!